

Come realizzare colori personalizzati [2]

Per predisporre **colori personalizzati** si possono impiegare queste tre funzioni:

→ **rgb(red, green, blue, alpha)** che genera i colori regolando il livello di intensità dei tre colori fondamentali rosso (*red*), verde (*green*) e blu (*blue*) e la trasparenza del colore (*alpha*);

→ **hsv(h, s, v, alpha)** che genera i colori regolando colore (*h*), saturazione (*s*), luminosità (*v*) e trasparenza (*alpha*);

→ **hcl(h, c, l, alpha)** che genera i colori regolando colore (*h*), cromaticità (*c*), luminanza (*l*) e trasparenza (*alpha*).

La funzione **rgb()** è trattata estesamente in un post a parte [1], qui vediamo invece le altre due, e vediamo poi come combinare i risultati di tutte e tre in **palette personalizzate** ovvero tavolozze che si possono impiegare per applicare automaticamente colori differenti alle diverse componenti di un grafico.

Copiate e incollate nella `Console di R` questo script e premete ↵ `Invio`.

```
# PERSONALIZZAZIONE DEI COLORI con la funzione hsv()
# regolando colore h, saturazione s, luminosità v, trasparenza alpha
#
mydata <- rnorm(5000, mean=0, sd=1) # genera cinquemila valori con media=0, ds=1
windows() # apre e inizializza una nuova finestra grafica
par(mfrow=c(4,4)) # suddivisione la finestra grafica in 4 righe per 4 colonne
# colore h
hist(mydata, col=hsv(h=0.9, s=1.0, v=1.0))
hist(mydata, col=hsv(h=0.6, s=1.0, v=1.0))
hist(mydata, col=hsv(h=0.4, s=1.0, v=1.0))
hist(mydata, col=hsv(h=0.1, s=1.0, v=1.0))
# saturazione s
hist(mydata, col=hsv(h=0.6, s=0.9, v=1.0))
hist(mydata, col=hsv(h=0.6, s=0.6, v=1.0))
hist(mydata, col=hsv(h=0.6, s=0.4, v=1.0))
hist(mydata, col=hsv(h=0.6, s=0.1, v=1.0))
# luminosità v
hist(mydata, col=hsv(h=0.4, s=1.0, v=0.9, alpha=0.5))
hist(mydata, col=hsv(h=0.4, s=1.0, v=0.6, alpha=0.5))
hist(mydata, col=hsv(h=0.4, s=1.0, v=0.4, alpha=0.5))
hist(mydata, col=hsv(h=0.4, s=1.0, v=0.1, alpha=0.5))
# trasparenza alpha
hist(mydata, col=hsv(h=0.1, s=1.0, v=1.0, alpha=0.9))
hist(mydata, col=hsv(h=0.1, s=1.0, v=1.0, alpha=0.6))
hist(mydata, col=hsv(h=0.1, s=1.0, v=1.0, alpha=0.4))
hist(mydata, col=hsv(h=0.1, s=1.0, v=1.0, alpha=0.1))
#
```

Nella funzione **hsv(h, s, v, alpha)** gli argomenti:

→ colore (*h*)

→ saturazione (*s*)

→ luminosità (*v*)

→ trasparenza (*alpha*)

possono assumere solamente valori compresi tra 0 e 1.

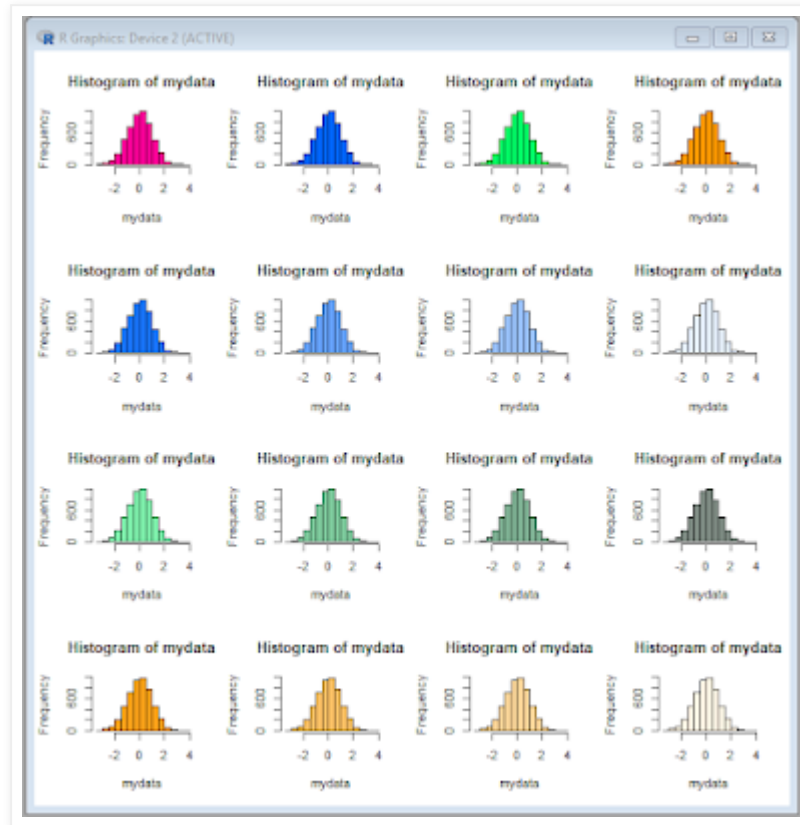
Allo scopo di illustrare i risultati forniti dalla funzione con istogrammi colorati:

→ nella prima riga di istogrammi è stato fatto variare l'argomento colore (*h*);

→ nella seconda riga è stato fatto variare l'argomento saturazione (*s*);

- nella terza riga è stato fatto variare l'argomento luminosità (v);
- nell'ultima riga è stato fatto varare l'argomento trasparenza (α).

Come si vede i colori della prima riga, con il massimo di saturazione e di luminosità e senza alcuna trasparenza, sono vividi. Gli altri colori di base possibili possono ovviamente essere visualizzati variando opportunamente i valori di (h), mentre si lasciano la combinazione dei diversi valori possibili per i quattro argomenti e i relativi risultati in termine di colore all'eventuale interesse e alla sperimentazione.



Da notare infine che i dati impiegati negli esempi sono generati ex-novo ogniqualvolta viene eseguito lo script, pertanto i grafici saranno sempre lievemente diversi gli uni dagli altri e da quelli qui riportati. Inoltre differenze più o meno importanti nel colore sono determinate dai monitor (per i quali esistono metodi di taratura in grado di ridurle) e nel caso delle stampe dal fatto che le stampanti impiegano la sintesi sottrattiva del colore (ciano, giallo e magenta sommati danno il nero).

Ora copiate e incollate nella Console di R questo script e premete ↵ Invio.

```

# PERSONALIZZAZIONE DEI COLORI con la funzione hcl()
# regolando colore h, cromaticità c, luminanza l e trasparenza alpha
#
mydata <- rnorm(5000, mean=0, sd=1) # genera cinquemila valori con media=0, ds=1
windows() # apre e inizializza una nuova finestra grafica
par(mfrow=c(4,4)) # suddivisione la finestra grafica in 4 righe per 4 colonne
# colore h
hist(mydata, col=hcl(h=60))
hist(mydata, col=hcl(h=120))
hist(mydata, col=hcl(h=240))
hist(mydata, col=hcl(h=300))
# cromaticità c
hist(mydata, col=hcl(h=120, c=100, l=40))
hist(mydata, col=hcl(h=120, c=100, l=60))
hist(mydata, col=hcl(h=120, c=100, l=80))
hist(mydata, col=hcl(h=120, c=100, l=90))
# luminanza l
hist(mydata, col=hcl(h=240, c=60, l=40))
hist(mydata, col=hcl(h=240, c=80, l=60))
hist(mydata, col=hcl(h=240, c=100, l=80))
hist(mydata, col=hcl(h=240, c=120, l=90))
# trasparenza alpha
hist(mydata, col=hcl(h=300, c=90, l=40, alpha=0.5))
hist(mydata, col=hcl(h=300, c=90, l=60, alpha=0.5))
hist(mydata, col=hcl(h=300, c=90, l=80, alpha=0.5))
hist(mydata, col=hcl(h=300, c=90, l=90, alpha=0.5))
#

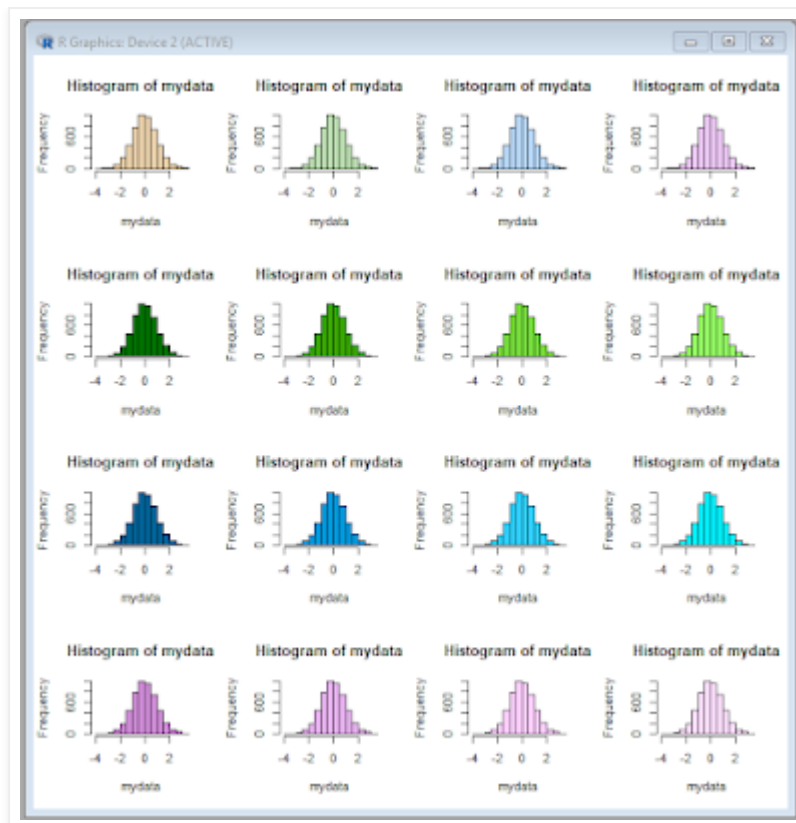
```

Nella funzione **hcl(h, c, l, alpha)** che genera i colori:

- il colore (**h**) può assumere valori compresi tra 0 e 360;
- il limite superiore della cromaticità (**c**) dipende dai valori di colore e luminanza;
- la luminanza (**l**) può assumere valori compresi tra 0 e 100;
- la trasparenza (**alpha**) può assumere valori compresi tra 0 e 1.

I colori della prima riga di istogrammi, che impiegano i valori di default `c=35` e `l=85`, mettono in evidenza la possibilità di ottenere con questa funzione colori pastello che è difficile trovare tra i 657 colori forniti nel pacchetto base di **R** [2].

Agendo su cromaticità (seconda riga) e luminanza (terza riga) si possono ottenere colori più saturi. Anche in questo caso si lasciano la combinazione dei diversi valori possibili per i quattro argomenti e i relativi risultati in termine di colore all'eventuale interesse e alla sperimentazione.



Una volta ottenuti i colori desiderati, si possono impiegare le funzioni che li hanno generati per ottenere i rispettivi codici esadecimali, che sono quelli che **R** impiega. Farlo è semplicissimo in quanto le tre funzioni per la generazione di colori, come avviene a titolo di esempio con queste tre righe di codice,

```
# generazione del codice esadecimale del colore
#
rgb(0.7, 0.2, 0.3, 0.5)
hsv(h=0.6, s=0.4, v=1.0)
hcl(h=120, c=100, l=60)
#
```

quando sono eseguite singolarmente

```
> rgb(0.7, 0.2, 0.3, 0.5)
[1] "#B3334D80"
> hsv(h=0.6, s=0.4, v=1.0)
[1] "#99C2FF"
> hcl(h=120, c=100, l=60)
[1] "#39A700"
```

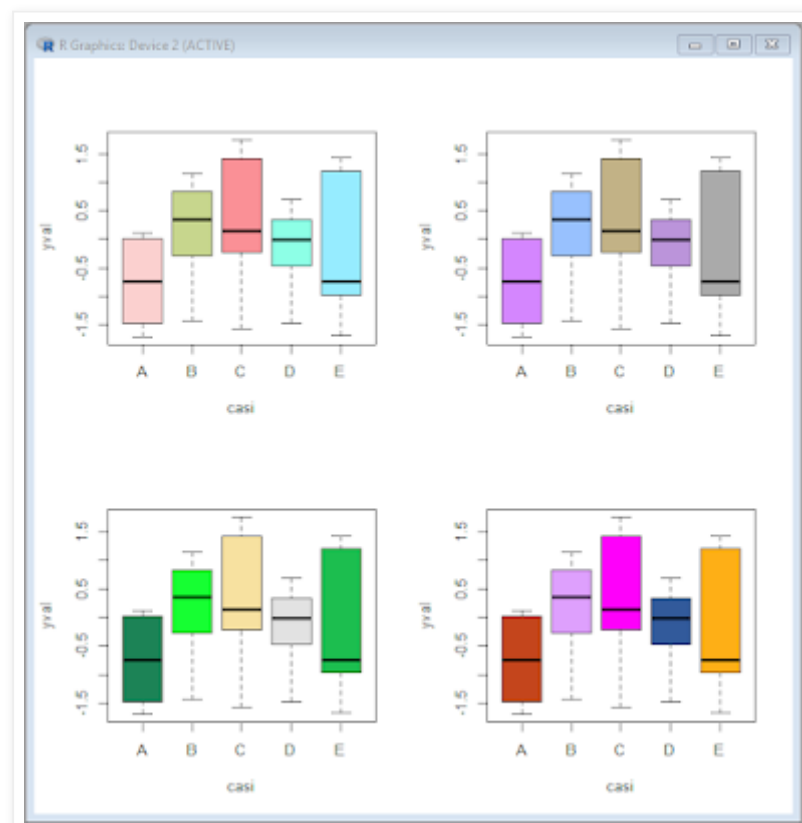
forniscono direttamente il codice esadecimale corrispondente, che può poi essere impiegato direttamente come valore dell'argomento **col**. Da notare che in questo esempio il codice esadecimale "#B3334D80" prodotto con la funzione **rgb()** ha due caratteri aggiuntivi (80) in quanto comporta anche la trasparenza (**0.5**) che non è stata prevista nelle altre due funzioni.

A questo punto i codici esadecimali (incluso il doppio apice che li racchiude) possono essere raccolti in tavolozze (**palette**) da impiegare per applicare automaticamente colori differenti alle diverse componenti di un grafico, come illustrato negli esempi che seguono.

Ora copiate e incollate nella Console di R questo script e premete ↵ Invio.

```
# PALETTE (TAVOLOZZE) PERSONALIZZATE
# boxplot differenziati in base a un fattore
#
mydata <- data.frame(xval=1:50, yval=rnorm(50), casi=rep(c("A", "B", "C", "D", "E"),
each=10)) # genera i dati di esempio
windows() # apre e inizializza una nuova finestra grafica
par(mfrow=c(2,2)) # suddivide la finestra grafica in 2 righe per 2 colonne
#
mypalette1 <- c("#F8A19F80", "#90AD1C80", "#F6222E80", "#1CFFCE80",
"#2ED9FF80") # tavolozza di colori con trasparenza al 50%
boxplot(yval~casi, data=mydata, horizontal=FALSE, notch=FALSE, col=mypalette1) #
traccia il boxplot
#
mypalette2 <- c("#AA0DFE80", "#3283FE80", "#85660D80", "#782AB680",
"#56565680") # tavolozza di colori con trasparenza al 50%
boxplot(yval~casi, data=mydata, horizontal=FALSE, notch=FALSE, col=mypalette2) #
traccia il boxplot
#
mypalette3 <- c("#1C8356", "#16FF32", "#F7E1A0", "#E2E2E2", "#1CBE4F") # tavolozza
di colori senza trasparenza
boxplot(yval~casi, data=mydata, horizontal=FALSE, notch=FALSE, col=mypalette3) #
traccia il boxplot
#
mypalette4 <- c("#C4451C", "#DEA0FD", "#FE00FA", "#325A9B", "#FEAF16") # tavolozza
di colori senza trasparenza
boxplot(yval~casi, data=mydata, horizontal=FALSE, notch=FALSE, col=mypalette4) #
traccia il boxplot
#
```

Come si vede a questo punto diventa semplice predisporre differenti combinazioni di colori e applicarle allo stesso grafico per valutarne preventivamente l'estetica, come nel caso di questo grafico a scatola con i baffi (boxplot).

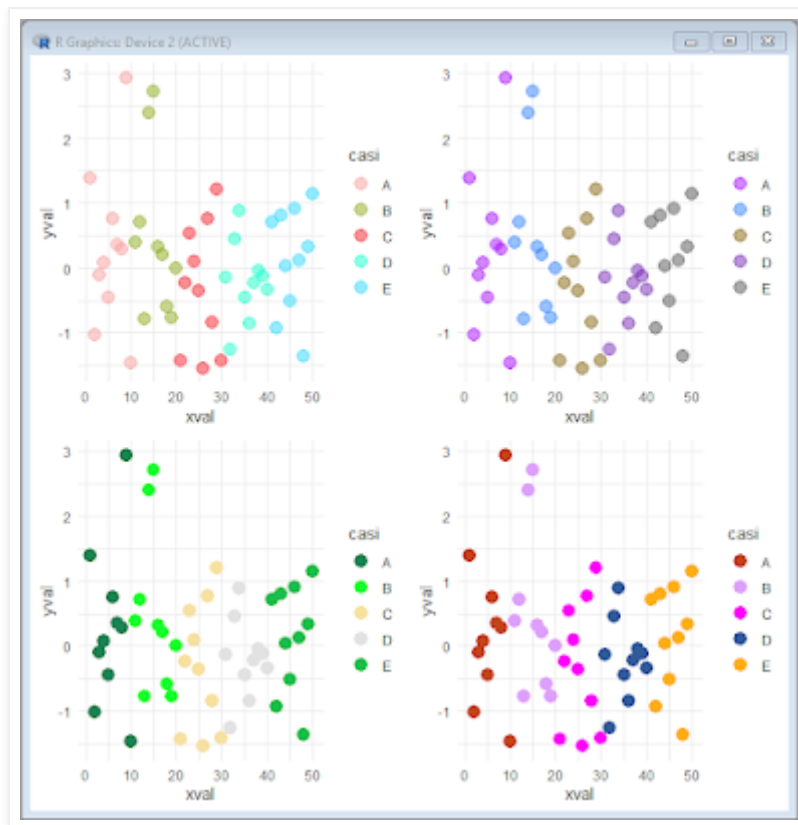


Ancor più interessante può essere l'applicazione delle stesse combinazioni di colori impiegate qui sopra a un grafico di dispersione (scatterplot) realizzato con *ggplot*.

Copiate e incollate nella `Console di R` questo script e premete ↵ Invio.

```
# PALETTE (TAVOLOZZE) PERSONALIZZATE con ggplot
# scatterplot differenziati in base a un fattore
#
library(ggplot2) # carica il pacchetto per la grafica
library(gridExtra) # carica il pacchetto per combinare i grafici in una sola figura
#
mydata <- data.frame(xval=1:50, yval=rnorm(50), casi=rep(c("A", "B", "C", "D", "E"),
each=10)) # genera i dati di esempio
windows() # apre e inizializza una nuova finestra grafica
#
mypalette1 <- c("#F8A19F80", "#90AD1C80", "#F6222E80", "#1CFFCE80",
"#2ED9FF80") # tavolozza di colori con trasparenza al 50%
plot1 <- ggplot(mydata, aes(x=xval, y=yval, color=casi)) + geom_point(size=4) +
scale_color_manual(values=mypalette1) + theme_minimal()
#
mypalette2 <- c("#AA0DFE80", "#3283FE80", "#85660D80", "#782AB680",
"#56565680") # tavolozza di colori con trasparenza al 50%
plot2 <- ggplot(mydata, aes(x=xval, y=yval, color=casi)) + geom_point(size=4) +
scale_color_manual(values=mypalette2) + theme_minimal()
#
mypalette3 <- c("#1C8356", "#16FF32", "#F7E1A0", "#E2E2E2", "#1CBE4F") # tavolozza
di colori senza trasparenza
plot3 <- ggplot(mydata, aes(x=xval, y=yval, color=casi)) + geom_point(size=4) +
scale_color_manual(values=mypalette3) + theme_minimal()
#
mypalette4 <- c("#C4451C", "#DEA0FD", "#FE00FA", "#325A9B", "#FEAF16") # tavolozza
di colori senza trasparenza
plot4 <- ggplot(mydata, aes(x=xval, y=yval, color=casi)) + geom_point(size=4) +
scale_color_manual(values=mypalette4) + theme_minimal()
#
# combina i grafici in una sola figura
grid.arrange(plot1, plot2, plot3, plot4, nrow=2, ncol=2)
#
```

Questo è il risultato, nel quale si può apprezzare come l'impiego di colori possa essere più soddisfacente dell'impiego di simboli, soprattutto nel caso di colori per i quali è stato previsto un effetto di trasparenza, che consente una migliore rappresentazione dei singoli dati quando questi sono molto ravvicinati (vedasi ad esempio il secondo grafico in alto a destra).



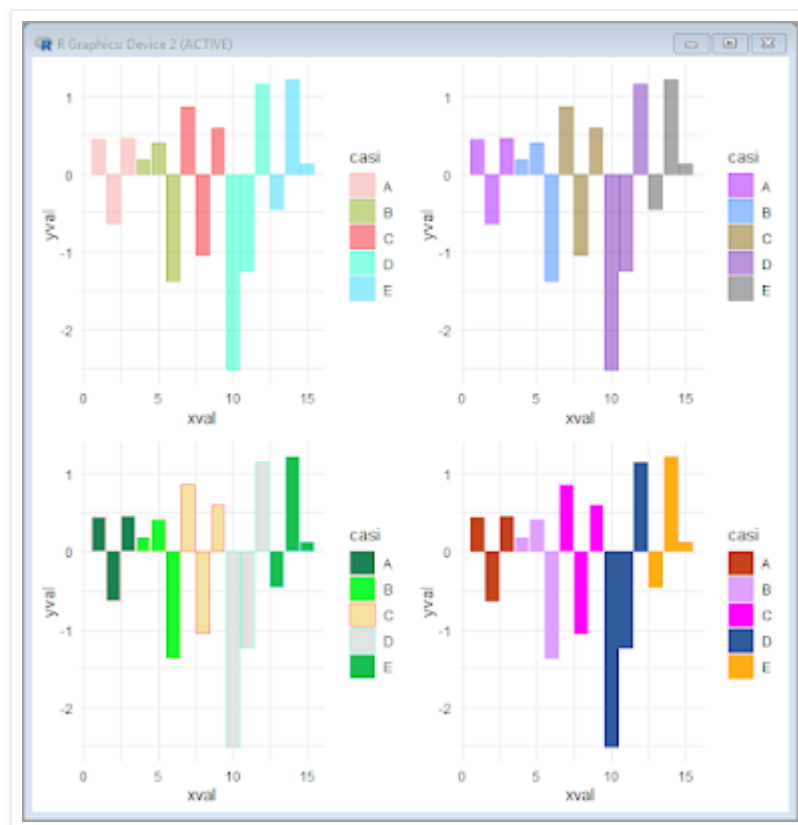
Ed ecco un ultimo esempio, un grafico a barre (barplot) realizzato con *ggplot* che mostra come anche nei grafici di questo tipo i colori possono essere impiegati per identificare i sottoinsiemi identificati da specifici fattori, sempre impiegando le palette precedenti. Copiate e incollate nella Console di R questo script e premete [↵](#) Invio.

```

# PALETTE (TAVOLOZZE) PERSONALIZZATE con ggplot
# barplot differenziati in base a un fattore
#
library(ggplot2) # carica il pacchetto per la grafica
library(gridExtra) # carica il pacchetto per combinare i grafici in una sola figura
#
mydata <- data.frame(xval=1:15, yval=rnorm(15), casi=rep(c("A", "B", "C", "D", "E"),
each=3)) # genera i dati di esempio
windows() # apre e inizializza una nuova finestra grafica
#
mypalette1 <- c("#F8A19F80", "#90AD1C80", "#F6222E80", "#1CFFCE80",
"#2ED9FF80") # tavolozza di colori con trasparenza al 50%
plot1 <- ggplot(mydata, aes(x=xval, y=yval, color=casi, fill=casi)) +
geom_bar(stat="identity") + scale_color_manual(values=mypalette1) +
scale_fill_manual(values=mypalette1) + theme_minimal()
#
mypalette2 <- c("#AA0DFE80", "#3283FE80", "#85660D80", "#782AB680",
"#56565680") # tavolozza di colori con trasparenza al 50%
plot2 <- ggplot(mydata, aes(x=xval, y=yval, color=casi, fill=casi)) +
geom_bar(stat="identity") + scale_color_manual(values=mypalette2) +
scale_fill_manual(values=mypalette2) + theme_minimal()
#
mypalette3 <- c("#1C8356", "#16FF32", "#F7E1A0", "#E2E2E2", "#1CBE4F") # tavolozza
di colori senza trasparenza
plot3 <- ggplot(mydata, aes(x=xval, y=yval, color=casi, fill=casi)) +
geom_bar(stat="identity") + scale_color_manual(values=mypalette3) +
scale_fill_manual(values=mypalette3) + theme_minimal()
#
mypalette4 <- c("#C4451C", "#DEA0FD", "#FE00FA", "#325A9B", "#FEAF16") # tavolozza
di colori senza trasparenza
plot4 <- ggplot(mydata, aes(x=xval, y=yval, color=casi, fill=casi)) +
geom_bar(stat="identity") + scale_color_manual(values=mypalette4) +
scale_fill_manual(values=mypalette4) + theme_minimal()
#
# combina i grafici in una sola figura
grid.arrange(plot1, plot2, plot3, plot4, nrow=2, ncol=2)
#

```

Questo è il risultato.



Conclusione: la realizzazione di **colori** personalizzati con le funzioni **rgb()**, **hsv()** e **hcl()** può rappresentare il punto di partenza per la loro combinazione in **palette** che possono essere salvate e riutilizzate al bisogno per dare un'impronta personale ai propri grafici.

[1] Vedere il post [Come realizzare colori personalizzati](#) [1]

[2] Vedere il post [Nomi e codici dei colori di R](#)